International Cybersecurity and Digital Forensics Academy (ICDFA)

Student Name	Ikpongakarase J. Anana
Registration Number	C11/26/PCDFIR
Course Code	CIP-B101
Course Title	Basic Computer Skills for Digital Forensic
Lab Title	Number Systems, ASCII, Timestamps and Data Representation
Lab Code	Lab1A
Date	November 4, 2026
Instructor	Basiru Aliyu

Tools Used

Bash terminal Python3 BC XXD Date md5sum / sha256sum

Practical Conversion Exercises

1. $101010_2 = 0 \times 2^0 = 0$
   $1 \times 2^1 = 2$
   $0 \times 2^2 = 0$
   $1 \times 2^3 = 8$
   $0 \times 2^4 = 0$
   $1 \times 2^5 = 32$
   42_{10}

2. $AB2_{16}$ $= 2 \times 16^0 = 2$
   A= 10 B=11 $11 \times 16^1 = 176$
   $10 \times 16^2 = 2560$
   2738_{10}

3. 126_{10}

2	126	
2	63	0
2	31	1
2	15	1
2	7	1
2	3	1
2	1	1
2	0	1

= **1111110_2**

4. 2738_{10} 16 | 2738
16 | 171 2
16 | 10 11 = B
16 | 0 10 = A
= **$AB2_{16}$**

101010110010_2 $0 \times 2^0 = 0$ $1 \times 2^0 = 1$ $0 \times 2^0 = 0$
$1 \times 2^1 = 2$ $1 \times 2^1 = 2$ $1 \times 2^1 = 2$
$0 \times 2^2 = 0$ $0 \times 2^2 = 0$ $0 \times 2^2 = 0$
$1 \times 2^3 = 8$ $1 \times 2^3 = 8$ $0 \times 2^3 = 0$
10 11 2
A B 2 = **$AB2_{16}$**

2.3 Validating with Linux Shell and Python3

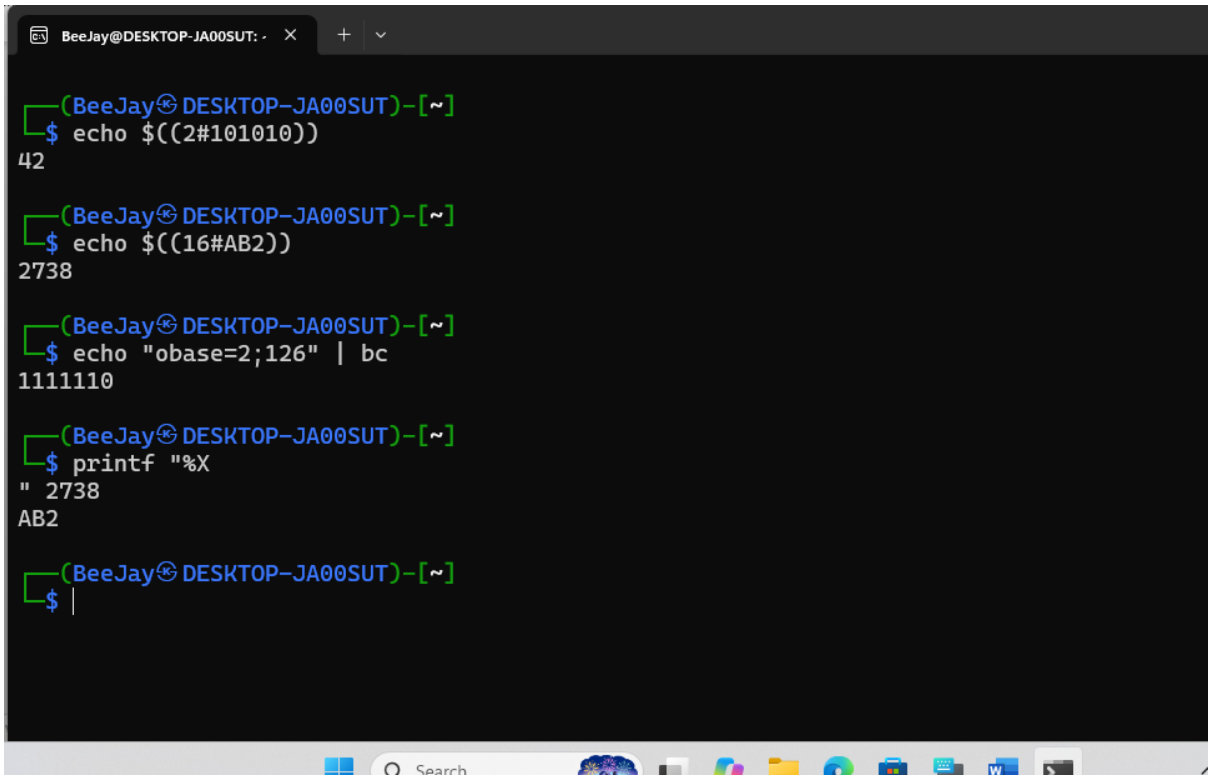


Fig2.3a Screenshot Showing Linux Shell Validation

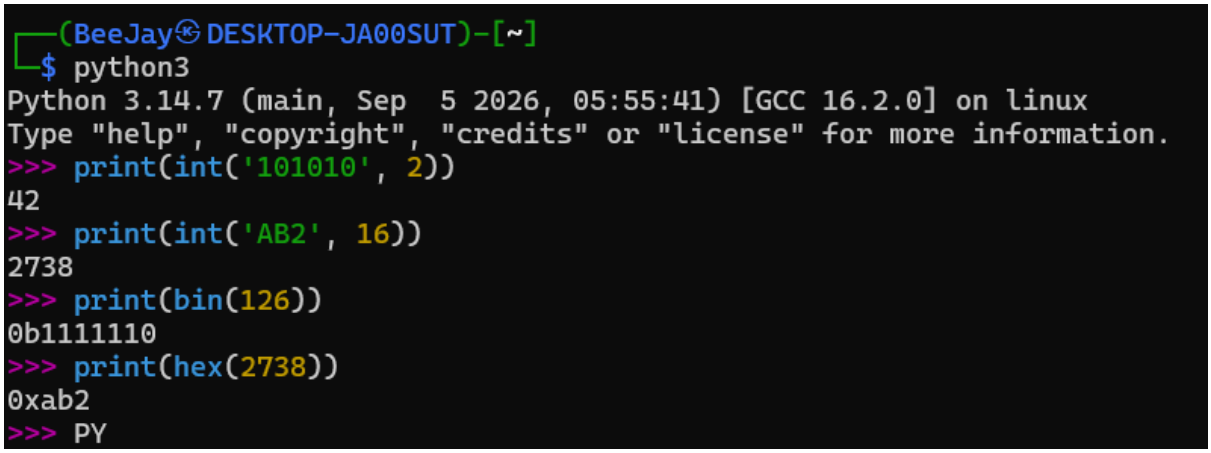


Fig2.3b Screenshot Showing Validation Using Python3

Commands

```
# Binary to decimal using shell arithmetic
echo $((2#101010))
# Hexadecimal to decimal using shell arithmetic
echo $((16#AB2))
# Decimal to binary using bc
echo "obase=2;126" | bc
# Decimal to hexadecimal using printf
printf "%X
" 2738

# Python validation
python3 -
print(int('101010', 2))
print(int('AB2', 16))
print(bin(126))
print(hex(2738))
```

2.4 Evidence Table to Complete

Evidence Value	Given Base	Manual Answer	Command Used to Verify	Screenshot Ref.
101010	Binary	42_{10}	echo $((2#101010))	Fig 2.3a
AB2	Hexadecimal	2738_{10}	echo $((16#AB2))	Fig 2.3a
126	Decimal	1111110_{2}	echo "obase=2;126" | bc	Fig 2.3a
2738	Decimal	$AB2_{16}$	printf "%X " 2738	Fig 2.3a
101010110010	Binary	$AB2_{16}$	hex(int('101010110010', 2))	Fig2.3b

3. Conversion Between Text and Hexadecimal Representations

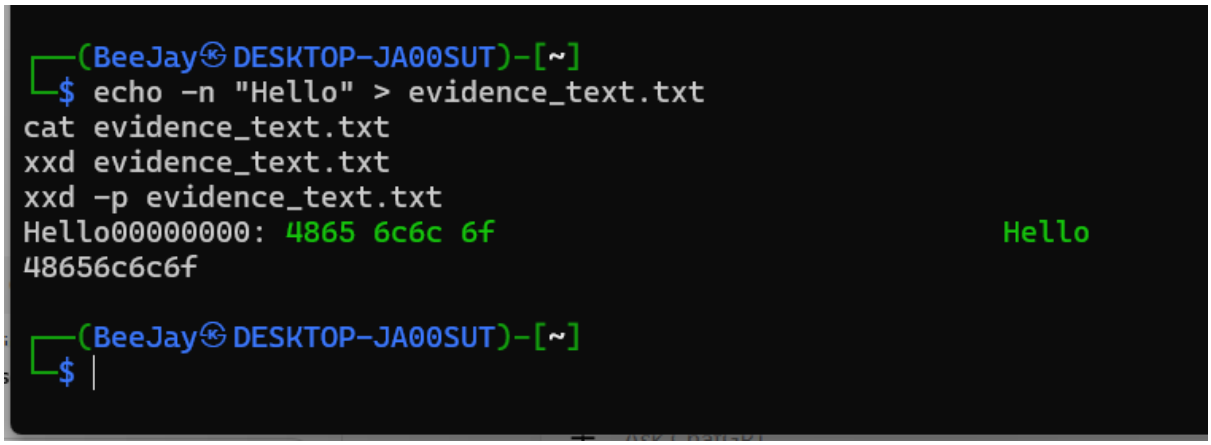


Fig3.1 Screenshot ASCII Evidence Text to View Hex

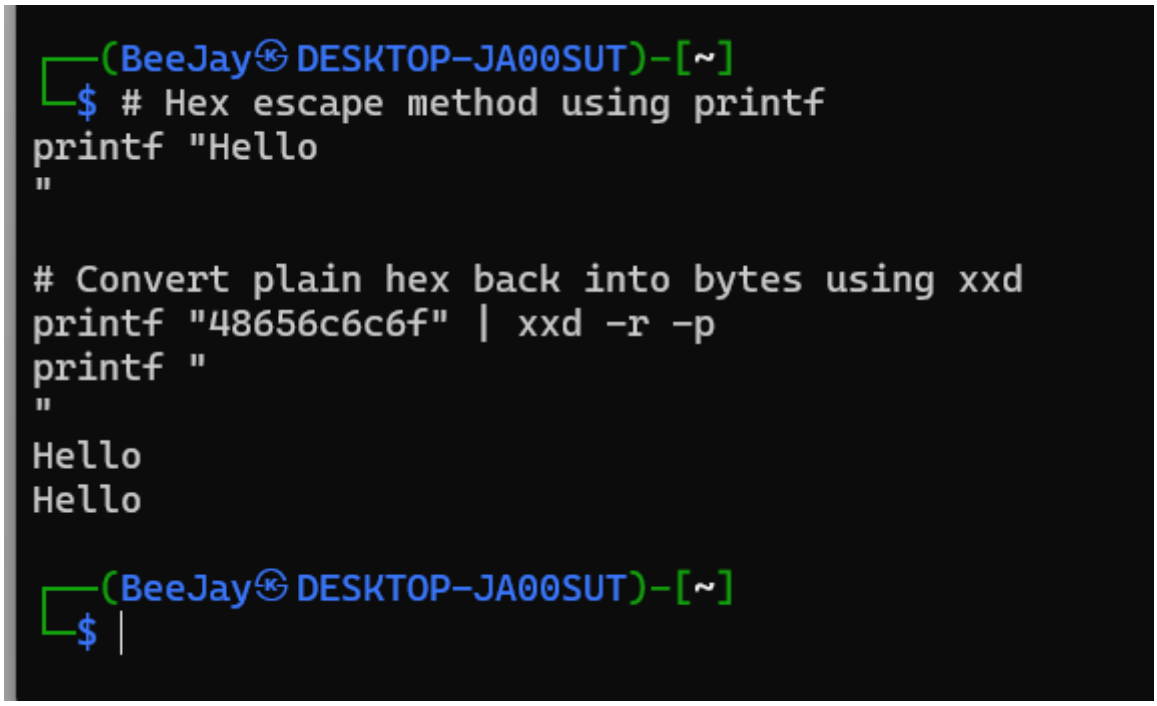


Fig3.2 Screenshot Hex to ASCII

```
┌──(BeeJay㉿DESKTOP-JA00SUT)-[~]
└─$ python3
Python 3.14.7 (main, Sep  5 2026, 05:55:41) [GCC 16.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import binascii
... text = b'Hello'
... print('ASCII bytes:', text)
... print('ASCII to hex:', binascii.hexlify(text).decode())
... print('Hex to ASCII:', binascii.unhexlify('48656c6c6f').decode())
... print('ord(H):', ord('H'))
... print('chr(72):', chr(72))
...
ASCII bytes: b'Hello'
ASCII to hex: 48656c6c6f
Hex to ASCII: Hello
ord(H): 72
chr(72): H
>>>
```

Fig3.3 Screenshot Showing Python3 Conversion

Code used in 3.4 Student Exercise

echo -n "Digital Forensics Begins with Bytes" > Ikpongakarase_lab1a.txt
xxd Ikpongakarase_lab1a.txt
xxd -p Ikpongakarase_lab1a.txt > hex_output.txt
cat hex_output.txt
xxd -r -p hex_output.txt

```
BeeJay@DESKTOP-JA00SUT: ~

┌──(BeeJay㉿DESKTOP-JA00SUT)-[~]
└─$ echo -n "Digital Forensics Begins with Bytes" > Ikpongakarase_lab1a.txt

xxd Ikpongakarase_lab1a.txt

xxd -p Ikpongakarase_lab1a.txt > hex_output.txt

cat hex_output.txt

xxd -r -p hex_output.txt
00000000: 4469 6769 7461 6c20 466f 7265 6e73 6963  Digital Forensic
00000010: 7320 4265 6769 6e73 2077 6974 6820 4279  s Begins with By
00000020: 7465 73                                  tes
4469676974616c20466f72656e7369637320426567696e73207769746820
4279746573
Digital Forensics Begins with Bytes
┌──(BeeJay㉿DESKTOP-JA00SUT)-[~]
└─$
```

Fig3.4 Screenshot showing Student Exercise

Why does 0A appear at the end when echo is used without -n?

The hexadecimal value 0A represents the Line Feed (LF) character in ASCII, when the echo command is used without the -n option, Linux automatically adds a newline character at the end of the text. This newline is represented by 0A in hexadecimal, so adding the -n option prevents this additional newline character from being added.

Therefore:

- echo "Hello" produces 48656c6c6f0a.
- echo -n "Hello" produces 48656c6c6f.

This distinction is important in digital forensics because even an additional newline changes the actual bytes and file contents.

A digital forensic examiner should not assume that every hexadecimal value represents readable text because hexadecimal is simply a numerical representation of raw bytes stored in a digital file. These bytes may represent different types of information. For example, when an examiner uses `xxd` to inspect a file, the hexadecimal output displays the underlying byte values. However, these values do not necessarily represent ordinary ASCII characters. Hexadecimal is a way of representing bytes, not proof that the underlying data is readable text. Misinterpreting binary data as text could lead to incorrect forensic conclusions. Therefore, an examiner must identify the file type, understand its structure and use appropriate forensic tools before interpreting the hexadecimal data.

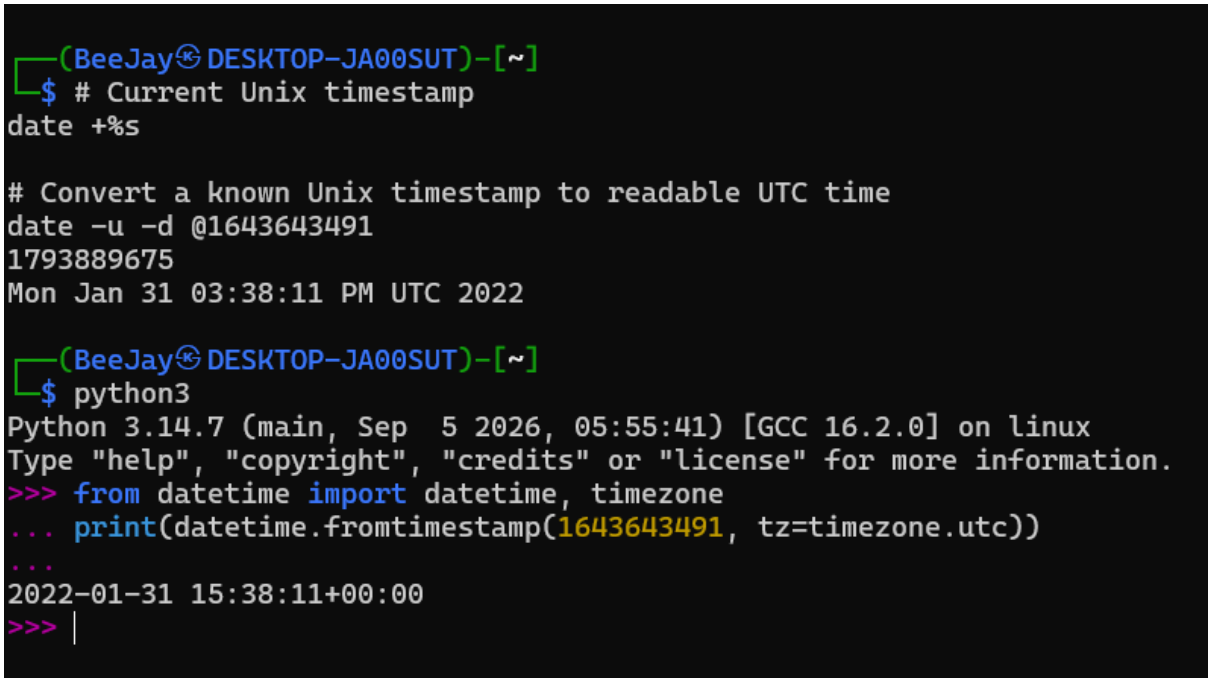


Fig4.1 Screenshot Showing Unix Epoch Conversion

```
┌──(BeeJay㉿DESKTOP-JA00SUT)-[~]
└─$ date
date -u
Thu Nov  5 03:44:40 PM WAT 2026
Thu Nov  5 02:44:41 PM UTC 2026

┌──(BeeJay㉿DESKTOP-JA00SUT)-[~]
└─$ python3
Python 3.14.7 (main, Sep  5 2026, 05:55:41) [GCC 16.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from datetime import datetime, timezone
... print('Local:', datetime.now())
... print('UTC  :', datetime.now(timezone.utc))
...
Local: 2026-11-05 15:45:21.718956
UTC  : 2026-11-05 14:45:21.719897+00:00
>>>
```

Fig4.2 Screenshot Comparing Local Time and UTC

```
BeeJay@DESKTOP-JA00SUT: ~
>>> from datetime import datetime, timezone
... unix_epoch = datetime(1970, 1, 1, tzinfo=timezone.utc)
... mac_epoch = datetime(2001, 1, 1, tzinfo=timezone.utc)
... print(int((mac_epoch - unix_epoch).total_seconds()))
...
... # Convert a sample Mac Absolute Time value to UTC by adding the offset
... mac_value = 725846400
... unix_value = mac_value + 978307200
... print(datetime.fromtimestamp(unix_value, tz=timezone.utc))
...
978307200
2024-01-02 00:00:00+00:00
>>>
```

Fig4.3 Screenshot Showing Mac Absolute Time

4 Timestamp Worksheet

Timestamp Value	Format	Conversion Command	Readable UTC Result	Screenshot Ref.
1643643491	Unix seconds	date -u -d @1643643491	Mon Jan 31 03:38:11 PM UTC 2022	Fig4.4
1675008832	Unix seconds	date -u -d @1675008832	Sun Jan 29 04:13:52 PM UTC 2023	Fig4.4
725846400	Mac Absolute Time seconds	date -u -d @$(python3 -c "print(725846400 + 978307200)")	Tue Jan 2 12:00:00 AM UTC 2024	Fig4.4
Current time	Unix seconds	date -u +%s	date -u +%s	Fig4.4

Table4.4 Showing Timestamp Worksheet

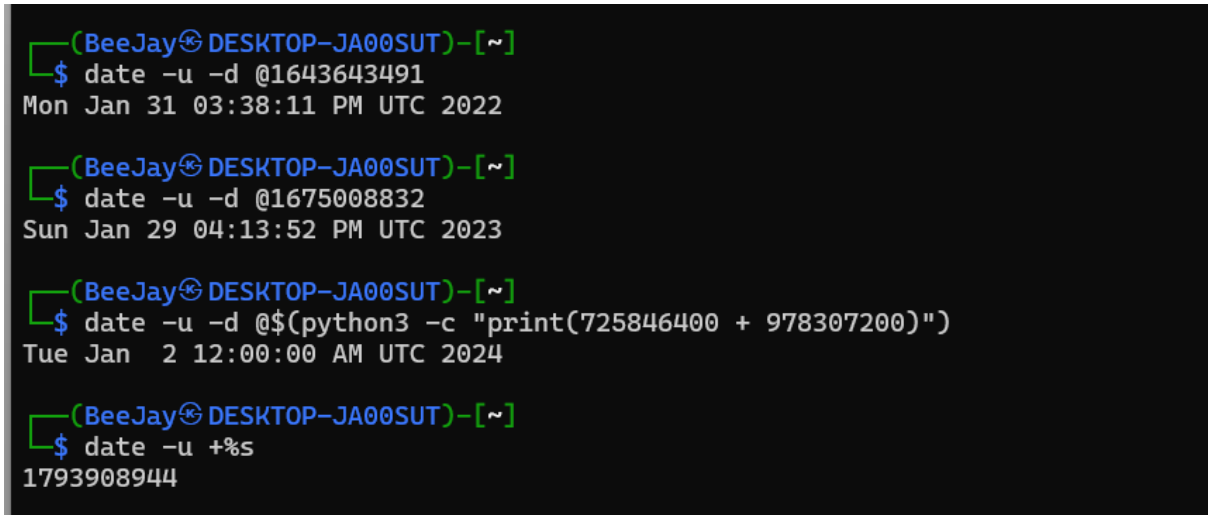


Fig4.4 Showing Timestamp Worksheet

5. Hashing and The Newline Problem

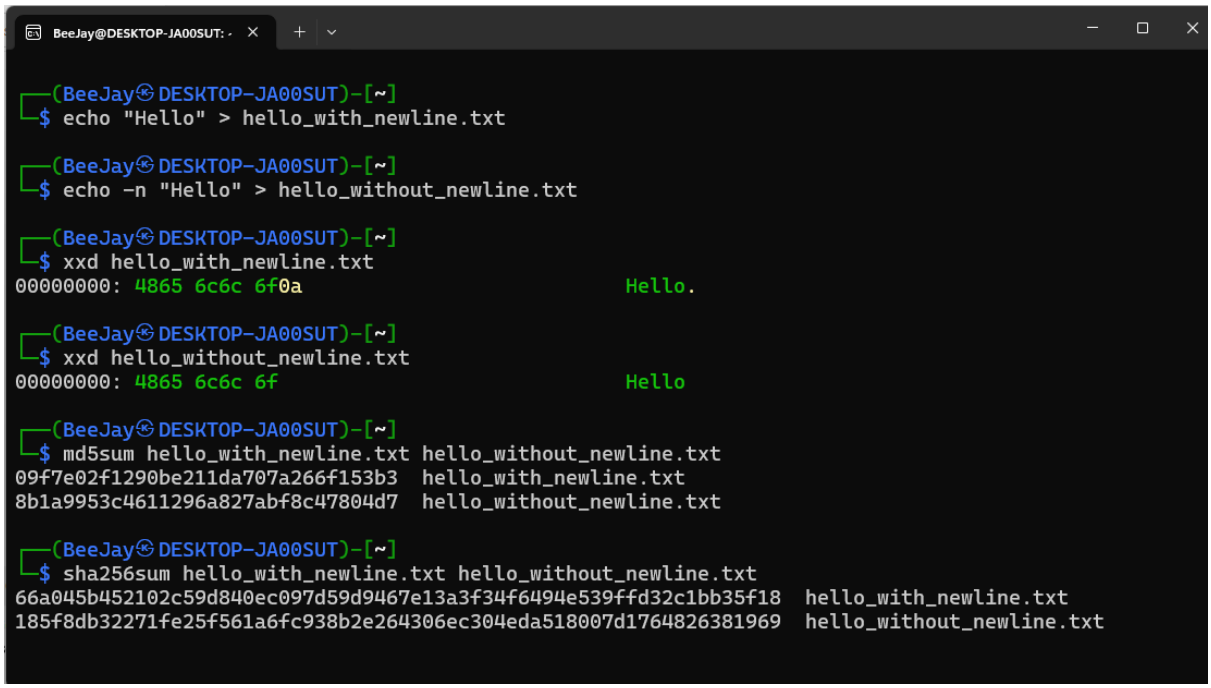


Fig5.1 Screenshot Showing Similar Files with Underlying Difference

5.2 Explain the Difference

Examining Fig5.1 above

- File 1 contains the character h e l l o with a new line byte
- File 2 contains the character h e l l o without the new line byte
- The files are not byte for byte identical even though they visibly look the same
- The newline character (0A) makes the first file different from the second at the byte level, resulting in different MD5 and SHA-256 hash values.

5.3 Student Exercise

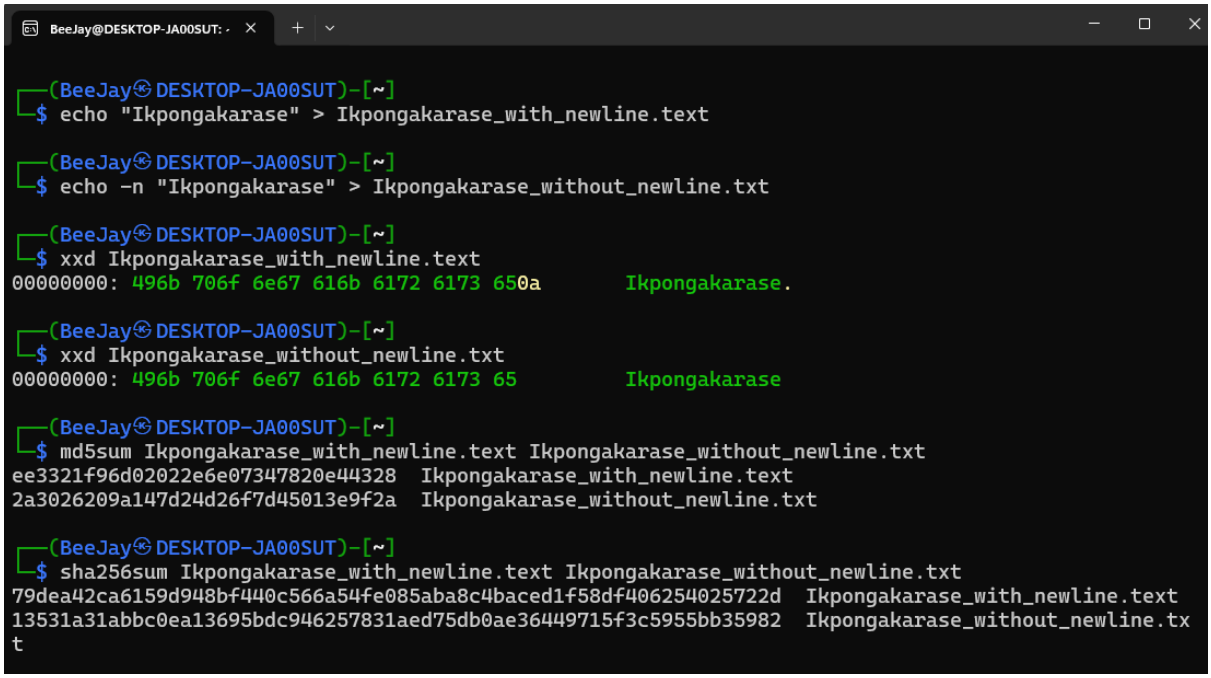


Fig5.3 Screenshot Showing My First Name Difference

The hash values differ because cryptographic hashes are byte-sensitive, and the two files containing my first name are different in the byte level, one of them contains the newline byte, so the MD5 and SHA-256 values are different.

The command I will use to hash only the visible characters are as seen below

1. echo -n "Ikpongakarase" | md5sum
2. echo -n "Ikpongakarase" | sha256sum

6. Endianness and Data Stored in Memory

The Figure below shows the check byte order and how to interpret the same bytes in two ways

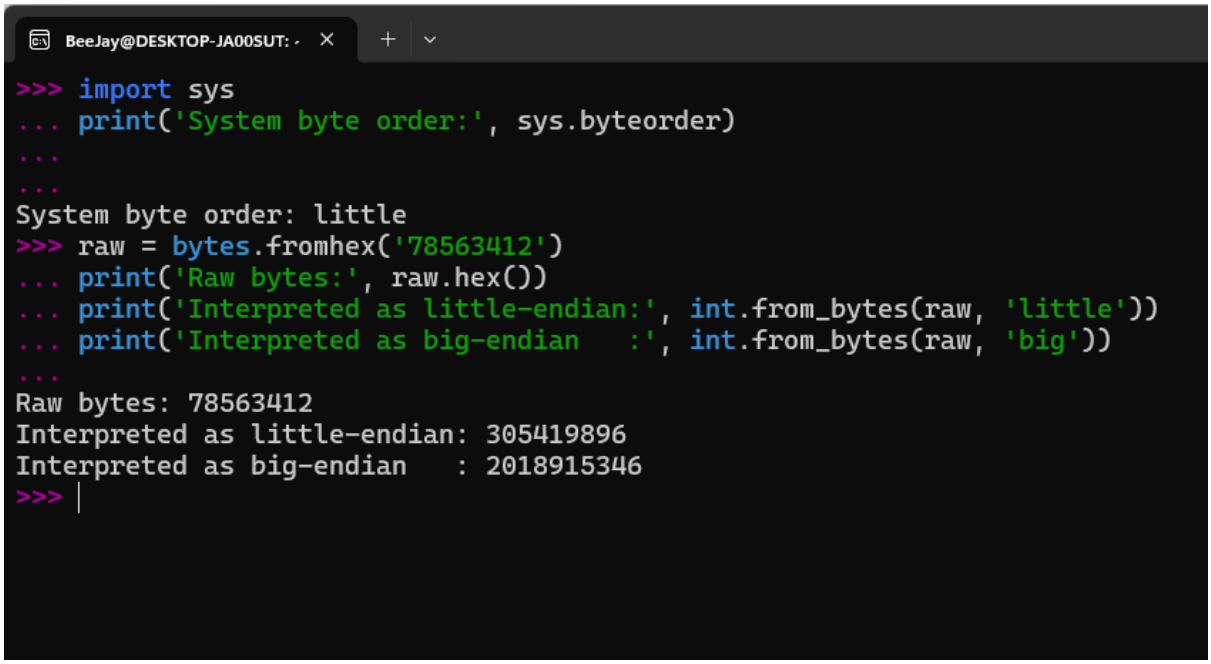


Fig6.1 Screenshot Showing Check Byte Order and Two Ways Interpretation

6.3 Student Exercise

Raw Hex Bytes	Little-Endian Decimal	Big-Endian Decimal	Command Used	Explanation
01000000	1	16,777,216	raw = bytes.fromhex('01000000') print(int.from_bytes(raw, 'little')) print(int.from bytes(raw, big))	Commands were entered individually and outputs obtained as seen in Fig6.3 below.
FF000000	255	4,278,190,080	raw = bytes.fromhex('FF000000') print(int.from_bytes(raw, 'little')) print(int.from bytes(raw, big))	Commands were entered individually and outputs obtained as seen in Fig6.3 below.
00000100	65,536	256	raw = bytes.fromhex('00000100') print(int.from_bytes(raw, 'little')) print(int.from bytes(raw, big))	Commands were entered individually and outputs obtained as seen in Fig6.3 below.
78563412	305,419,896	2,018,915,346	raw = bytes.fromhex('78563412') print(int.from_bytes(raw, 'little')) print(int.from bytes(raw, big))	Commands were entered individually and outputs obtained as seen in Fig6.3 below.

Table6.3 Student Exercise



Fig6.3 Screenshot Showing Student Exercise.

Explanation

The experiment demonstrated that byte order affects how binary data is interpreted. Using Python's int.from_bytes() function, the same raw bytes produced different decimal values when interpreted as Little-Endian and Big-Endian. This highlights the importance of identifying the correct byte order when analysing digital evidence.

Reflection: Number systems and timestamps are important in digital forensics because they help investigators understand, decode and interpret digital evidence accurately. Number systems such as binary, decimal and hexadecimal allow investigators to examine raw data, identify file structures and recover readable information from digital devices. Timestamps help establish when files were created, modified or accessed, making it possible to reconstruct events and develop accurate timelines during an investigation. Understanding different timestamp formats, byte orders and time zones also prevents misinterpretation of evidence. Together, these techniques help ensure the accuracy, reliability and integrity of digital forensic investigations.